

Distributed Systems

Lecture7: Symmetrically Distributed System Architectures

Mohammad Reza Shakournia

shakournia@ut.ac.ir

School of Electrical and Computer Engineering,
University of Tehran,
Tehran, Iran

Vertical/Horizontal Distribution

- **Vertical distribution:** In many business environments, distributed processing is equivalent to organizing a client-server application as a multitiered architecture.
- It is achieved by placing logically different components on different machines.
- **Horizontal Distribution:** In this type of distribution, a client or server may be physically split up into logically equivalent parts, but each part is operating on its own share of the complete data set, thus balancing the load.
- Peer-to-peer systems supports horizontal distribution.
- The processes that constitute a peer-to-peer system are all equal.
- As a consequence, much of the interaction between processes is symmetric:
  - each process will act as a client and a server at the same time.

Structured Peer-to-peer Systems

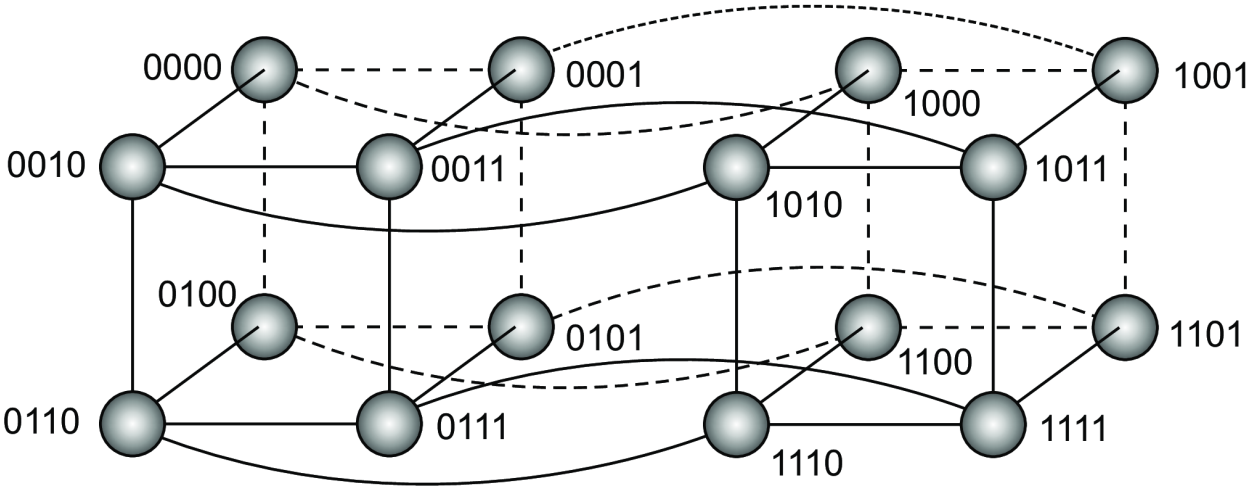
- In a structured peer-to-peer system the nodes are organized in an overlay that adheres to a specific, deterministic topology.
- This topology is used to efficiently look up data.
- Each data item that is to be maintained by the system, is uniquely associated with a key, and that this key is subsequently used as an index.

$$key(data\ item) = hash(data\ item's\ value).$$

- P2P system now responsible for storing (key,value) pairs.

Example: Hypercube

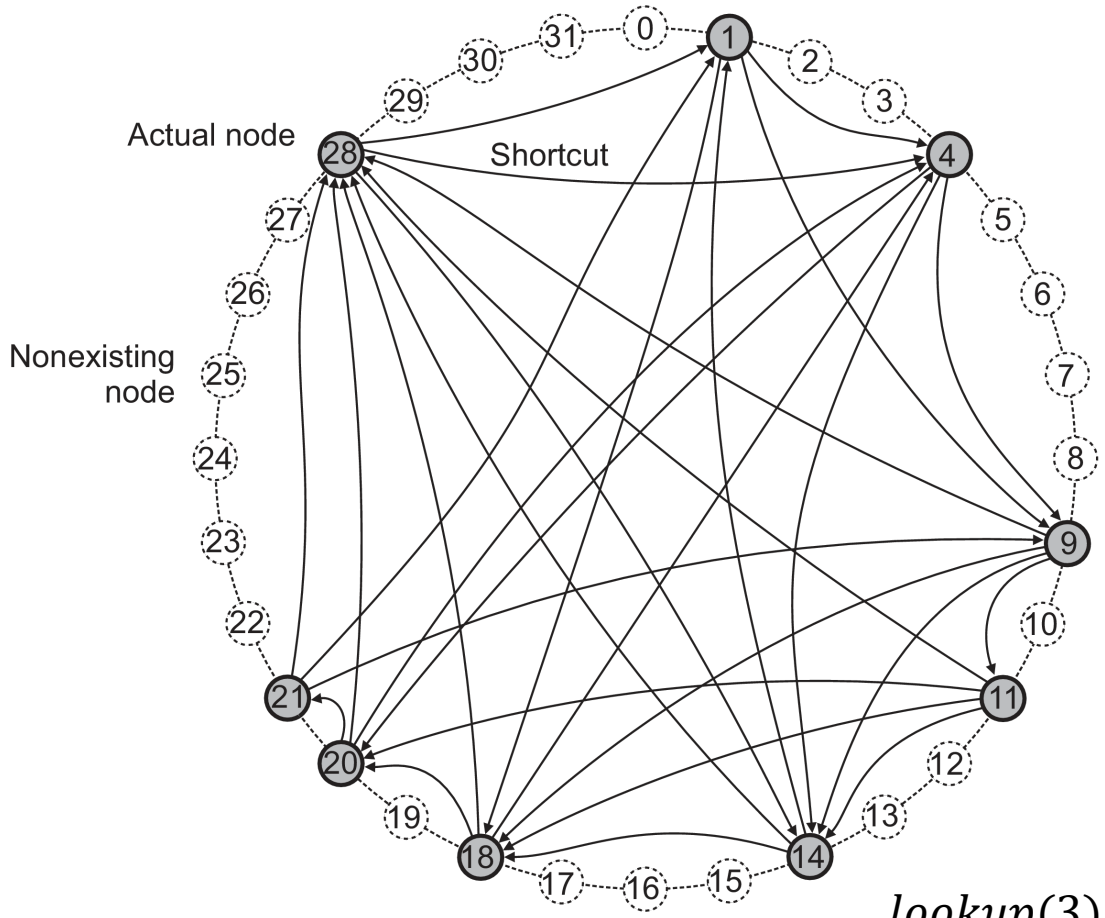
- Looking up d with key $k \in \{0,1,2,\ldots,2^4 - 1\}$ means routing request to node with identifier k.



Example: Chord

- Nodes are logically organized in a ring. Each node has an m-bit identifier.
- Each data item is hashed to an m-bit key.
- Data item with key k is stored at node with smallest identifier $id \geq k$, called the successor of key k.
- The ring is extended with various shortcut links to other nodes.

Example: Chord



$lookup(3)@9: \ 28 \rightarrow 1 \rightarrow 4$

Unstructured Peer-to-peer Systems

- Each node maintains an ad hoc list of neighbors. The resulting overlay resembles a random graph: an edge $\langle u, v \rangle$ exists only with a certain probability $P[\langle u, v \rangle]$.
- Looking up:
  - **Flooding:** issuing node u passes request for d to all neighbors. Request is ignored when receiving node had seen it before. Otherwise, v searches locally for d (recursively). May be limited by a Time-To-Live: a maximum number of hops.
  - **Random walk:** issuing node u passes request for d to randomly chosen neighbor, v. If v does not have d, it forwards request to one of its randomly chosen neighbors, and so on.

Flooding vs Random Walk

- Assume N nodes and that each data item is replicated across r randomly chosen nodes.
- **Random Walk:**
  - $P[k]$ probability that item is found after k attempts:

$$P[k] = \frac{r}{N}(1 - \frac{r}{N})^{k-1}.$$

  - S ("search size") is expected number of nodes that need to be probed:

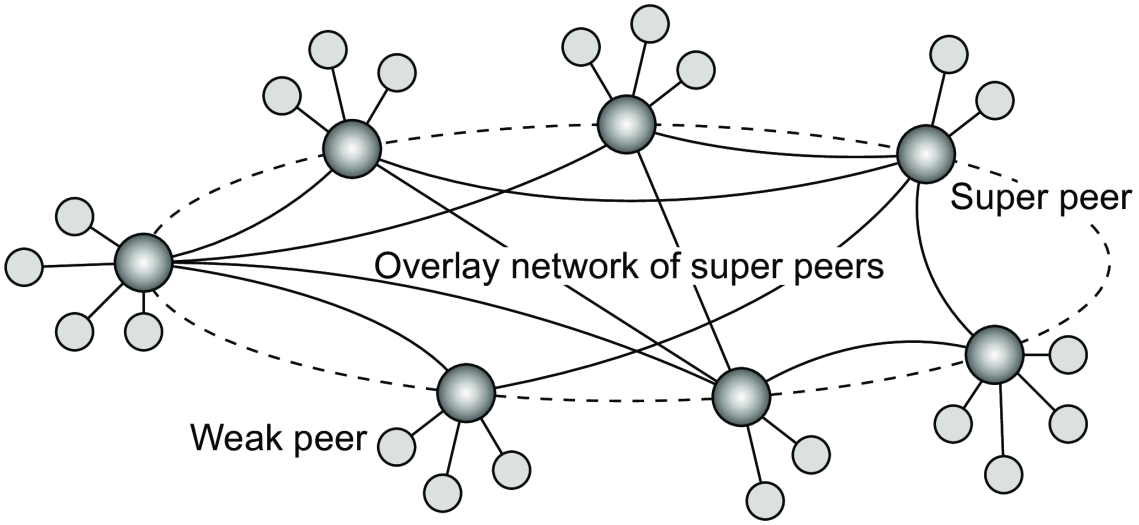
$$S = \sum_{k=1}^{N} k.P[k] = \sum_{k=1}^{N} k.\frac{r}{N}(1 - \frac{r}{N})^{k-1} \approx \frac{N}{r} \; for \; 1 \ll r \leq N.$$

Flooding vs Random Walk

- Assume N nodes and that each data item is replicated across r randomly chosen nodes.
- **Flooding:**
  - Flood to d randomly chosen neighbors.
  - After k steps, some $R(k) = d.(d-1)^{k-1}$ will have been reached (assuming k is small).
  - With fraction r/N nodes having data, if $\frac{r}{N}.R(k) \geq 1$, we will have found the data item.
- **Comparison:**
  - If $r/N = 0.001$, then $S \approx 1000$
  - With flooding and $d = 10, k = 4$, we contact 7290 nodes.
  - Random walks are more communication efficient, but might take longer before they find the result.

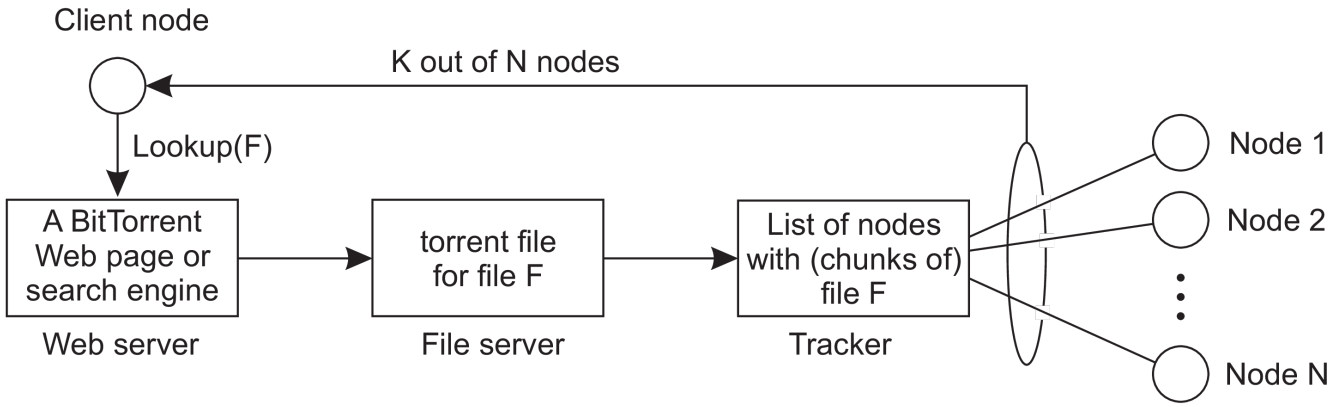
Super-peer Networks

- It is sometimes sensible to break the symmetry in pure peer-to-peer networks:
- When searching in unstructured P2P systems, having index servers improves performance.
- Deciding where to store data can often be done more efficiently through brokers.



The BitTorrent Case

- Search for a file F:
  - Lookup file at a global directory ⇒ returns a torrent file
  - Torrent file contains reference to tracker: a server keeping an accurate account of active nodes that have (chunks of) F.
  - P can join swarm, get a chunk for free, and then trade a copy of that chunk for another one with a peer Q also in the swarm.



Wrap-Up + What's Next

In the next lecture, we will discuss hybrid system architectures including:

- Cloud computing,
- Edge-cloud computing,
- Blockchain Architecture.